

Checkpoint & Re-spawning
Hinge Joint
Animated Particle Effect

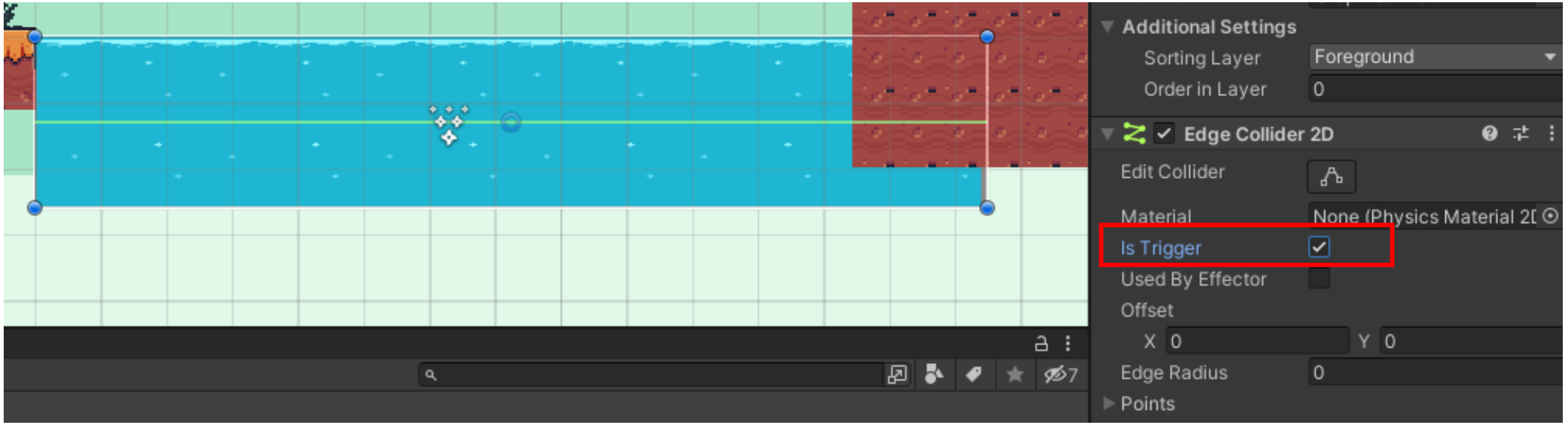


Checkpoints

- We need to take into account that players can fall into the water or, eventually, get knocked out by an enemy. Would we start them at the beginning of the level all over again? Of course not! We’re going to create a checkpoint system to account for such circumstances.

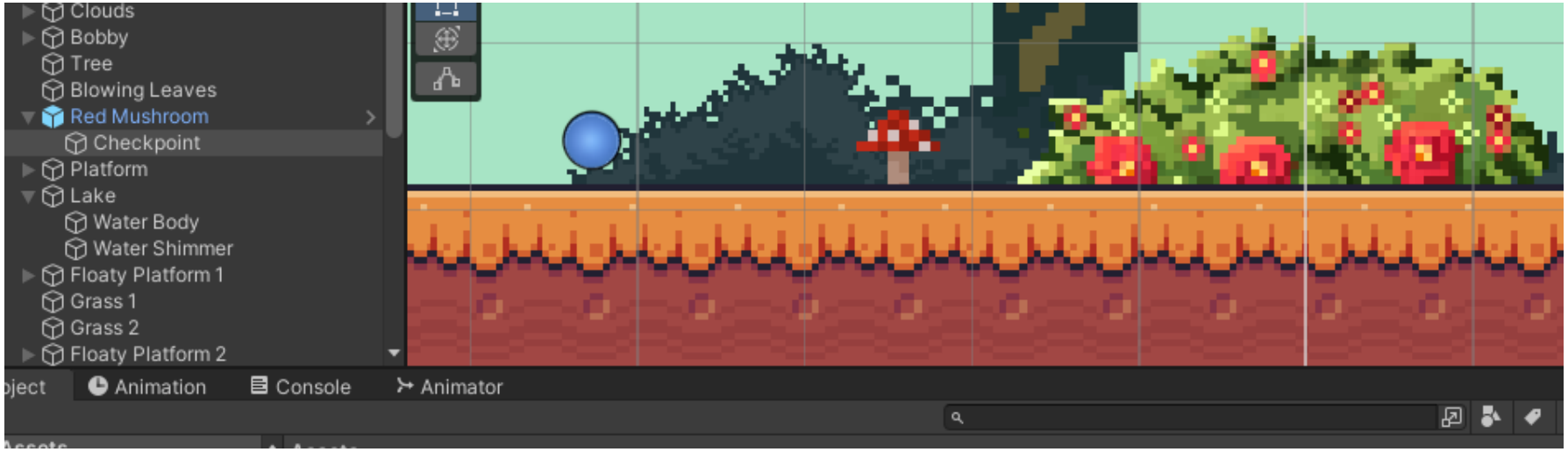
Exercise #15 - Checkpoints – Step 1: Make the Lake Deadly

- Make sure the Lake's Sorting Layer is on Foreground
- Add a Box Collider 2D or Edge Collider2D component to the lake
- Since we want a script to execute as soon as the Player touches the Lake's collider, make sure to check the IsTrigger property



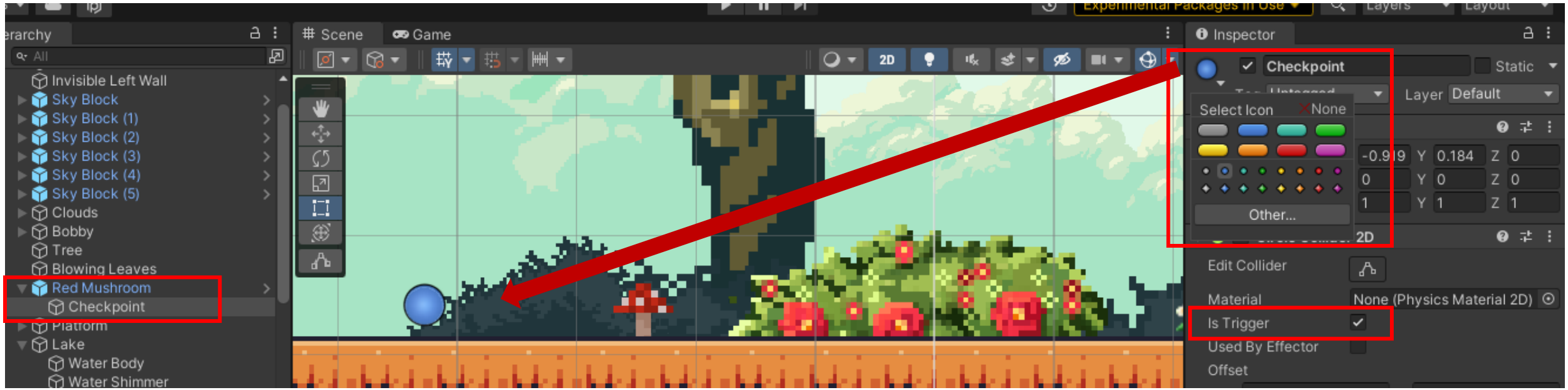
Exercise #15 - Checkpoints – Step 2: Create a Checkpoint

- Create a new game object (e.g. tree, mushroom, etc.) that's gonna be the location your player can respawn
- In our example, Player is supposed to respawn under the tree by the red mushroom
- Create a child for the mushroom and call it Checkpoint



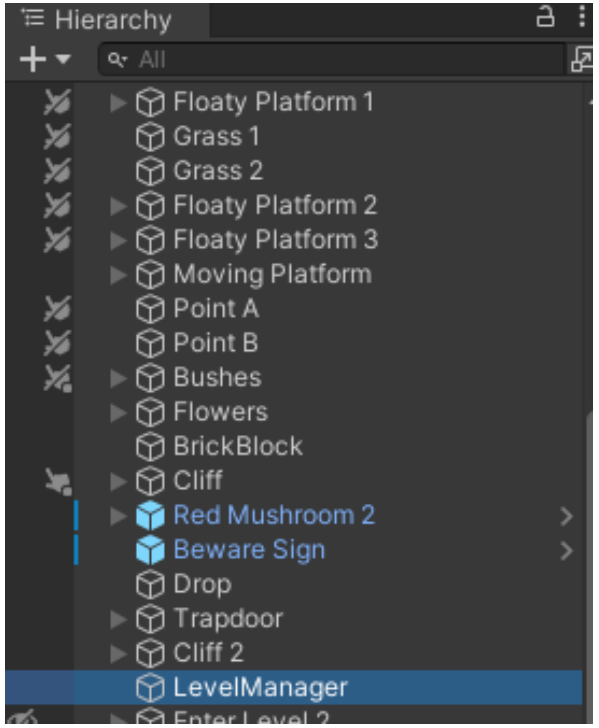
Exercise #15 - Checkpoints – Step 2: Create a Checkpoint

- Give the child a flag so you can see it. Place it close to the mushroom
- Add a Circle Collider 2D and make sure it's triggered



Exercise #15 - Checkpoints – Step 3: Add a Level Manager GameObject

- Create another empty game object to be used as a level manager, rename it to *Level Manager*
- Level Manager will be responsible for all logic related to this game level, but it doesn't need to be visible on the scene like Checkpoint and everything else



Exercise #15 - Checkpoints – Step 4: Add three script files

- **Attach scripts to each gameobject** as following:
  - **LevelManager** (**Script name:** LevelManager)
  - **Checkpoint** (**Script name:** Checkpoint)
  - **Lake** (**Script name:** DeadlyDrop)

Exercise #15 - Checkpoints – Step 6: Write LevelManager script Logic

- LevelManager script is responsible for tracking the current checkpoint, it will use the current checkpoint's position to return our player to it when respawned

```
public class LevelManager : MonoBehaviour
{
    2 references | Unity Serialized Field = null
    public GameObject CurrentCheckpoint; //so we can update the current checkpoint from within Unity

    // Use this for initialization
    0 references | Unity Message
    void Start () {

    CurrentCheckpoint = null;

    }

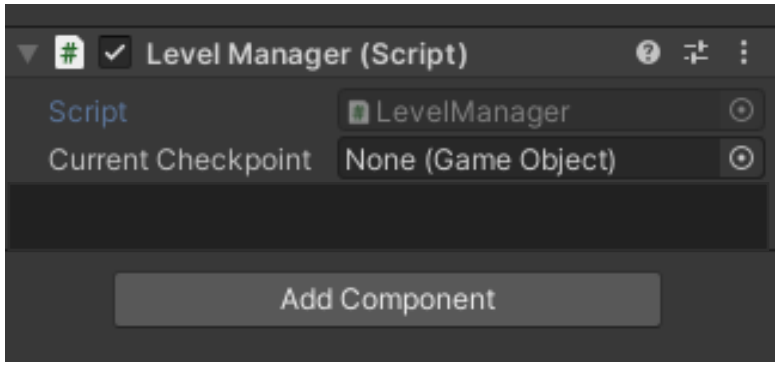
    // Update is called once per frame
    0 references | Unity Message
    void Update () {

    }

    0 references
    public void RespawnPlayer()
    {
        FindObjectOfType<PlayerController>().transform.position = CurrentCheckpoint.transform.position;//Search for the asset/object called
        //Controller (your player's script code name whatever it is). Once you've found it, change its player game object's position
        //to be at thew last checkpoint the player passed through before s/he died
    }
}
```

Exercise #15 - Checkpoints – Step 6: Write LevelManager script Logic

- There can be many checkpoints in the same level, right? So the CurrentCheckpoint variable should be always updating
- When the game first begins, its value is null, but every time the player passes by a new checkpoint, it should forget the old one and replace it with the new one





- Will start off like this

Then will get a value later

Exercise #15 - Checkpoints – Step 7: Write Checkpoint script logic

- Checkpoints script is responsible for updating the *currentCheckpoint* variable of the LevelManager when player passes by it
- The *FindObjectOfType()* function allows you to easily access public variables in other scripts and change their values

```
void OnTriggerEnter2D (Collider2D other)
{
    if (other.tag == "Player")
        //if the collider of the object whose name is Sonic GameObjects touches the checkpoint's circle collider


       FindObjectOfType<LevelManager>().CurrentCheckpoint = this.gameObject;
    //go to the Level Manager script, and update the value of CurrentCheckpoint to become the new Checkpoint the player has
    //just passed through. This is necessary when you have several checkpoints in a level
}
```

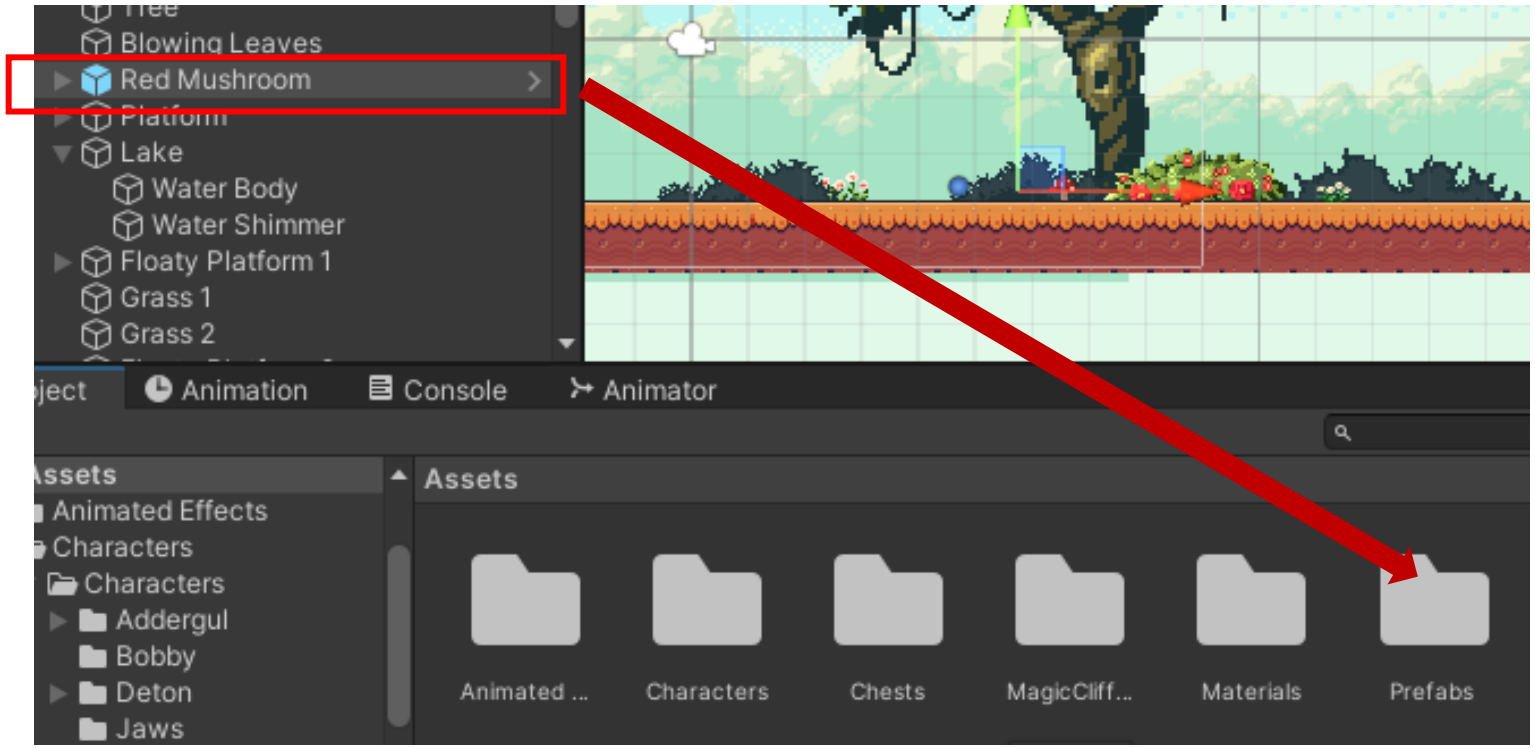
Exercise #15 - Checkpoints – Step 8: Create a Prefab (for multiple checkpoints)

- Now that the first checkpoint is ready and complete, we can create a template of it and reuse endlessly in the game. This saves a lot of time

- That template in Unity is called a **Prefab**

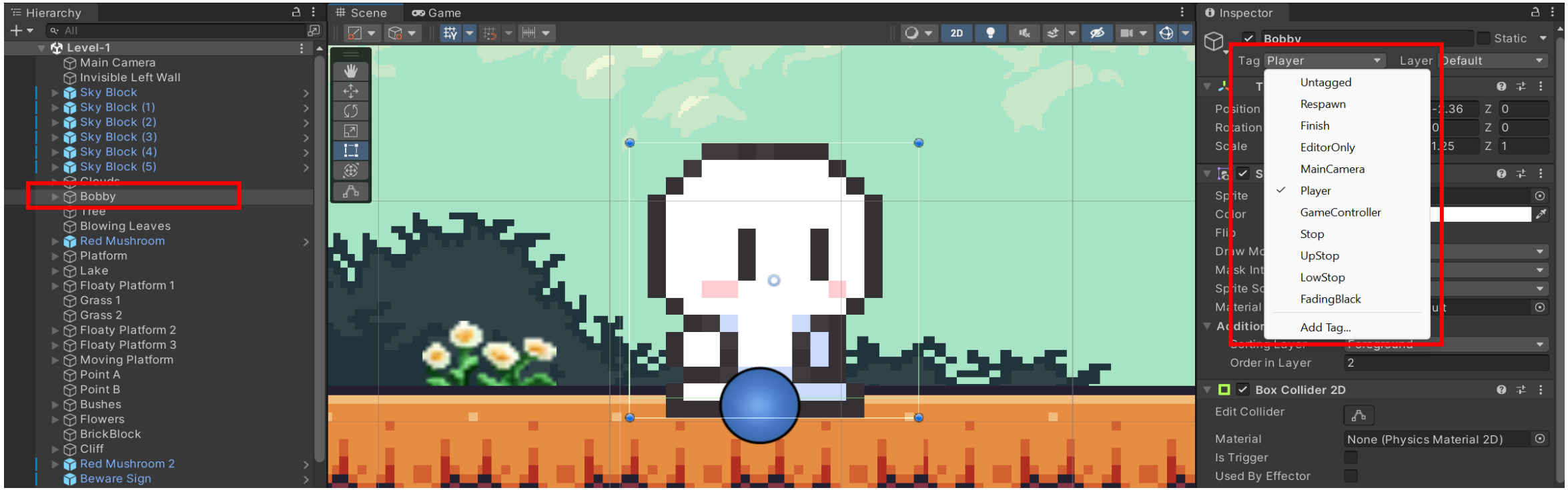
Exercise #15 - Checkpoints – Step 8: Create a Prefab (for multiple checkpoints)

- To create a prefab of our checkpoint (the mushroom), drag the game object from the Hierarchy down into the Prefabs folder in your Assets
- Notice that the object turns blue



Exercise #15 - Checkpoints – Step 9: Write Lake script Logic

- Give the Player a ‘Tag’ in the Inspector view. You will open the list of tag if you found Player tag choose it, if not add Tag “Player”. We’ll use it to make if-conditions easier



Exercise #15 - Checkpoints – Step 9: Write Lake script Logic

- When the Player falls into the lake, he's supposed to respawn at the last checkpoint he passed
- Create a script for the Lake and name it *DeadlyDrop*
- Add an OnTriggerEnter2D() function

```
//function that executes upon trigger (when something collides with the spikes)
void OnTriggerEnter2D(Collider2D other)
{
    if (other.tag == "Player")
    //if the collider of the object whose name is Sonic GameObjects touches the spike collider

     FindObjectOfType<LevelManager>().RespawnPlayer();
    //go to the Level Manager script, and execute the Respawn Player function
}
```

Exercise #15 - Checkpoints – Step 9: Write Lake script Logic

- Now play the scene. Let the player run through the checkpoint first, then fall into the lake. The moment he collides with the lake's collider, he'll immediately respawn back at the checkpoint under the tree

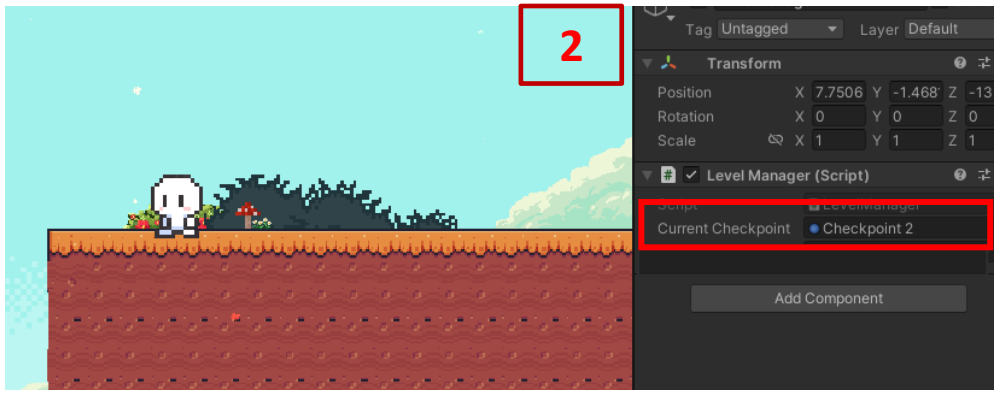


Exercise #15 - Checkpoints – Step 10: Add a Second Checkpoint

- Add a second checkpoint beyond the lake on top of the cliff.
- Notice how when the player runs by it, the currentCheckpoint variable in LevelManager will forget the first one under the tree, and replace it with the new one



1st checkpoint is under tree



2nd checkpoint is at cliff edge

Hinge Joint Constraint

- The most common use of a hinge joint in Unity is to create a *trapdoor* for your player character to fall through

- Unlike your player character, you don’t want your platform or ground to fall away due to gravity!

- So we add a hinge constraint to tell the physics engine that the object has a point on it that will constrain it to its position on the screen but allow it to rotate with the force of gravity without actually falling. There are a few different constraint types you are able to add to a Rigidbody2D Component.

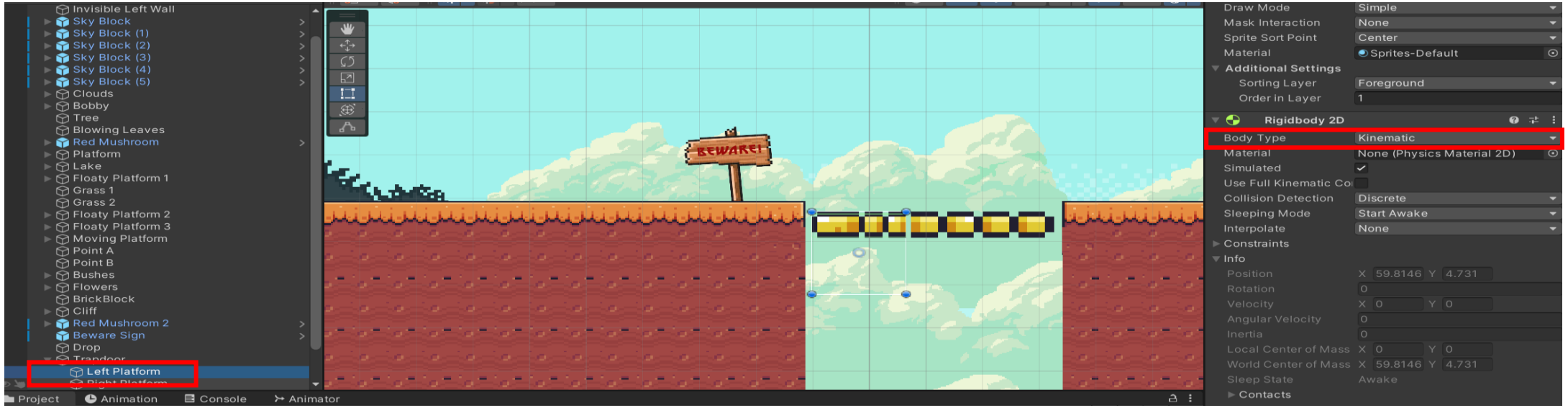
- A Hinge Joint can be triggered by a collision from another game object

Other Types of Joint Constraints

- There are other types of joints other than the Hinge Joint. Some of them are:
  - **Spring Joint 2D:** Creates a spring-type effect with two colliding GameObjects
  - **Slider Joint 2D:** Constrains a GameObject along a set path, like an elevator door. Can also be set to activate upon collision or continually
  - **Wheel Joint 2D:** Constrains an object to a position but allows it to rotate

Exercise #16 – Create a Hinge Joint

- Choose a couple platform game objects and attach a Rigidbody2D and BoxCollider2D to both of them.
- For the left Platform, we will change the body type of the Rigidbody2D to **Kinematic**



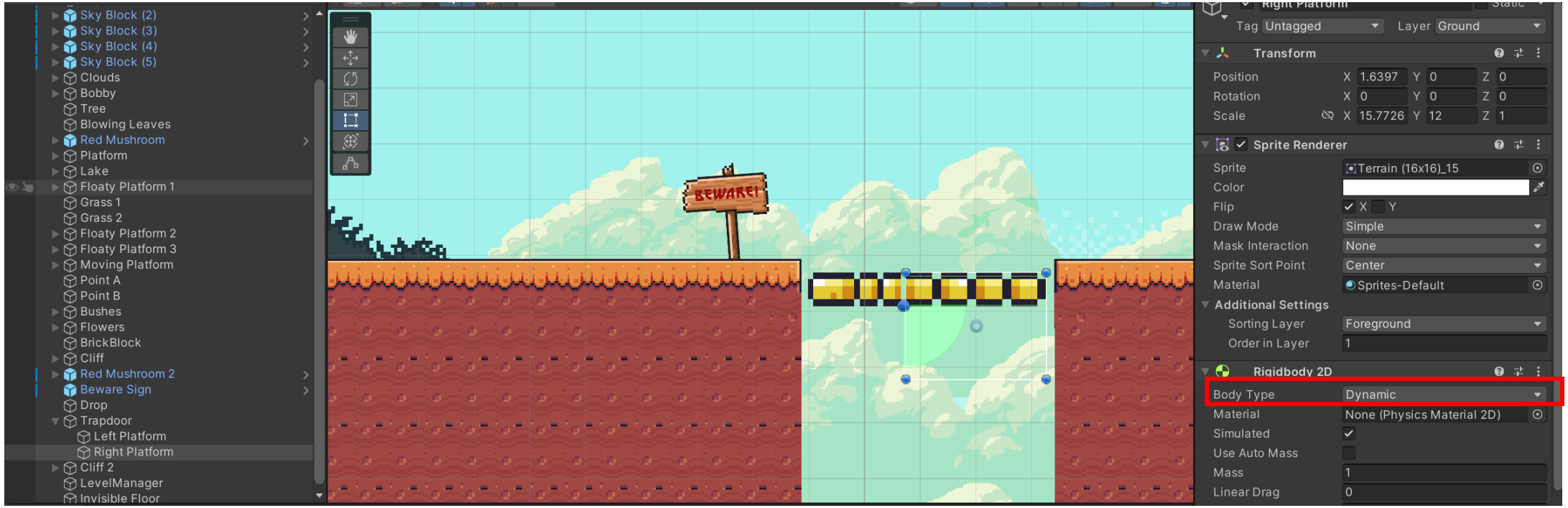
Exercise #16 – Create a Hinge Joint

What is the difference between the Rigidbody2D body types?

Body Type	Behavior
Dynamic	Moves under physics forces and gravity.
Kinematic	Moves under script or explicit control, not normal physics forces.
Static	Does not move and is intended for objects that remain fixed.

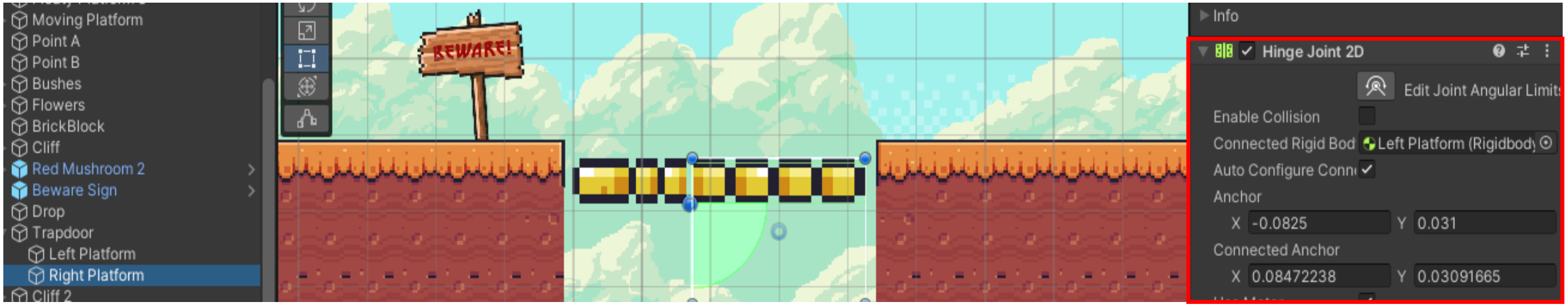
Exercise #16 – Create a Hinge Joint

- For the right Platform, we leave the body type of the Rigidbody2D **Dynamic** as we want this platform to be responding to gravity to move.



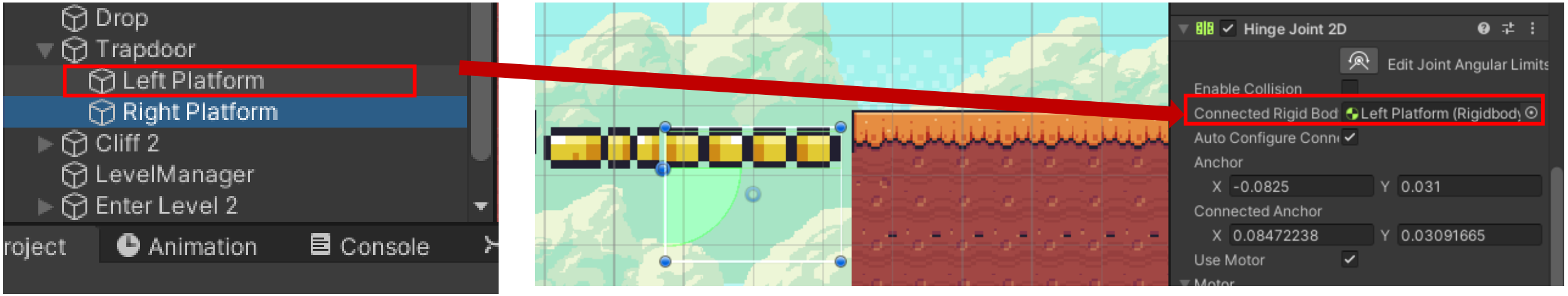
Exercise #16 – Create a Hinge Joint

- Attach a HingeJoint2D to the one on the **right only**
- In the example below, we want the one on the right to open *downwards* like a trap door when the player collides with it



Exercise #16 – Create a Hinge Joint (Cont.)

- In the Hinge Joint2D component of the **right-side platform** in Inspector, **change the value of the Connected Rigid Body property by dragging the platform game object on the left** from the Hierarchy view and dropping it into the text box in the Inspector view

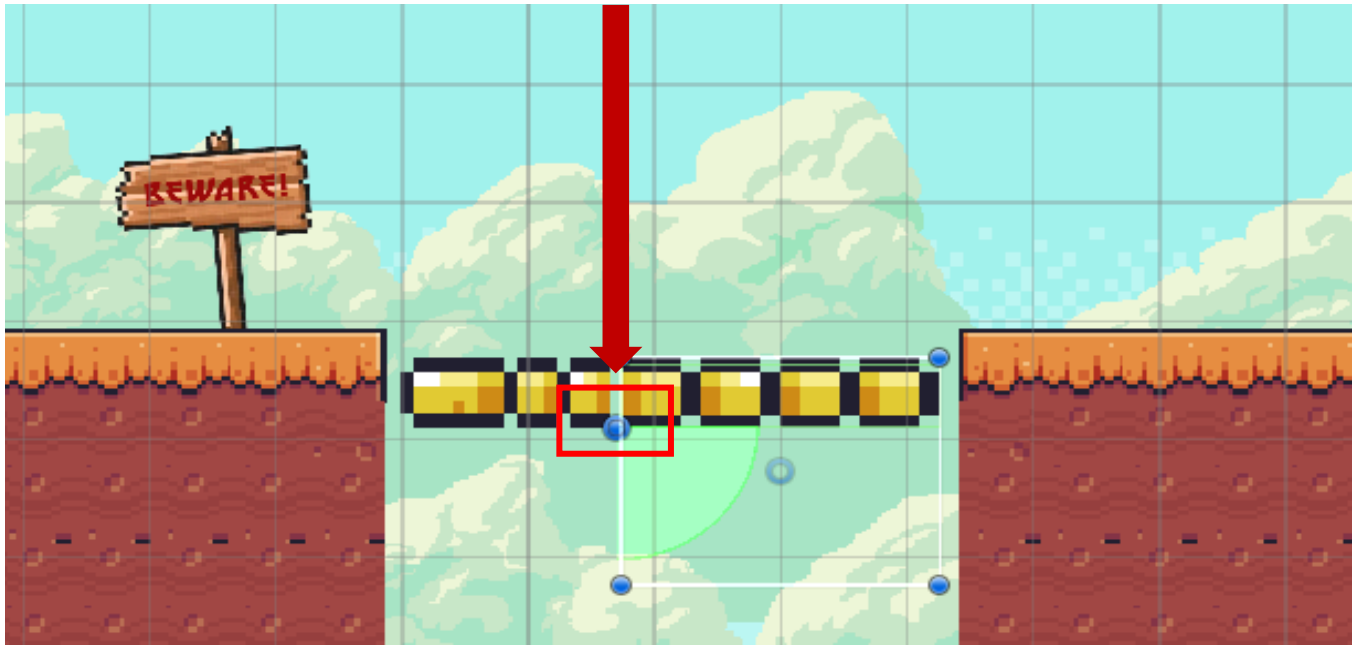


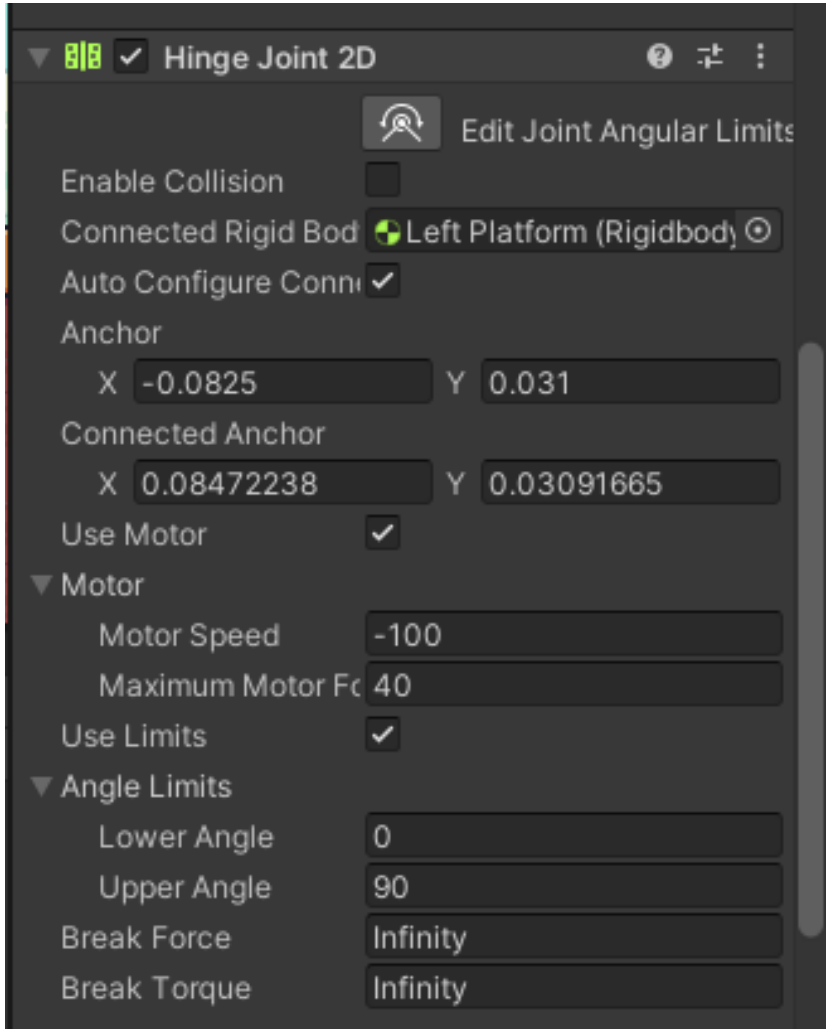
Exercise #16 – Create a Hinge Joint (Cont.)

- Understand what these properties do to get the hinge joint working the way you want it to:
  - **Anchor** – the hollow circle. It's the point your platform rotates around
  - **Connected Anchor** – the full circle. It indicates where in your scene the anchor is placed. If you're not sure where that is, experiment with its position till you get it the way you want
    - (In many cases, putting the full circle inside the hollow circle will be enough for you to get the effect you need)
  - **Enable Collision** – check this if you don't want your trapdoor to swing through the platform on the left as if it's invisible! You want them to *collide* with each other
  - **Use Limits** – check it if you don't want your trapdoor to rotate fully when something collides with it. Change the values of Lower Angle and Upper Angle until you get the effect you need. For this exercise, use degrees **0 and 90**, respectively
  - **Use Motor** – check it to apply torque and make the platform rotate and control speed of rotation when the player collides with it. For this exercise, use the values **-100 and 40**, respectively

Exercise #16 – Create a Hinge Joint (Cont.)

- Make sure that you have edited the hinge Point to be in between the two platforms.





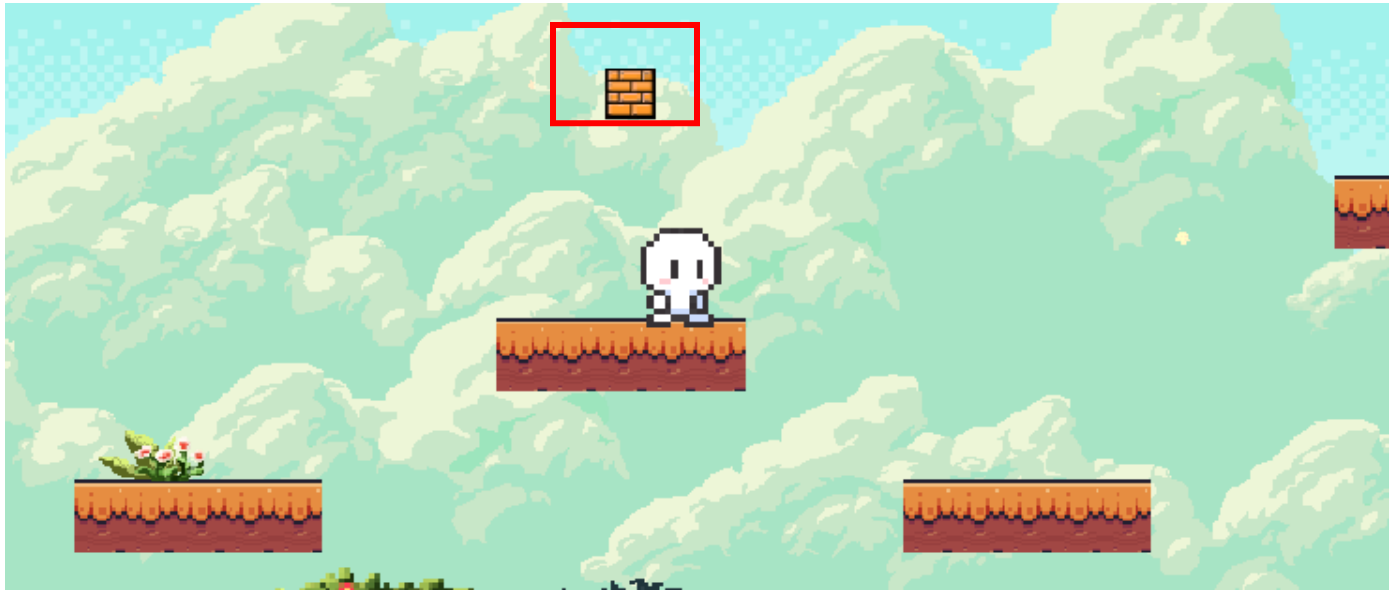
Exercise #17 – Particle Effect (Animated)

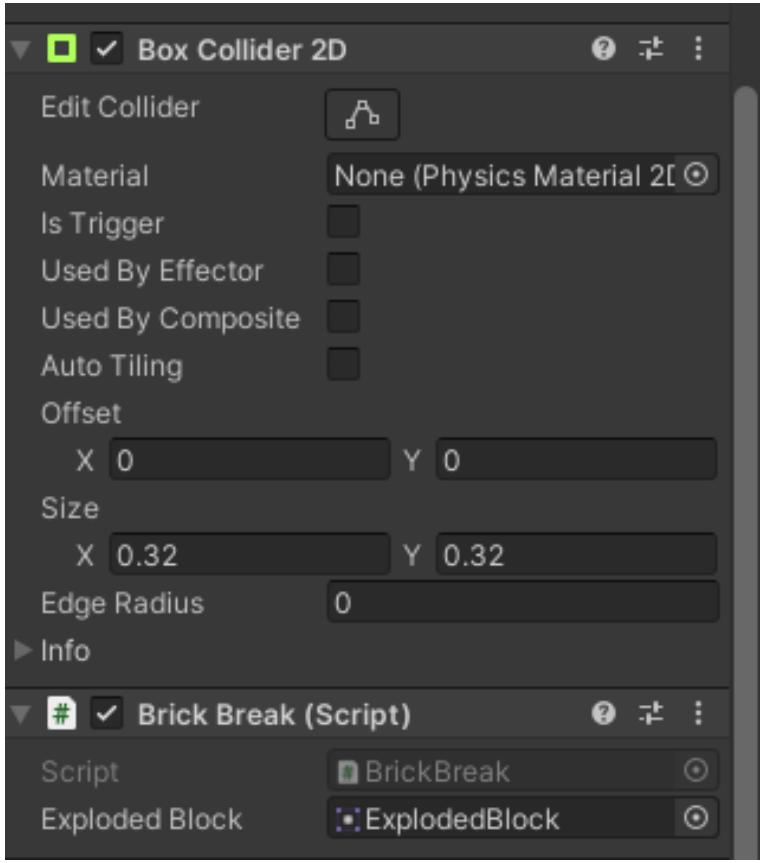
- Particle effect is a unique tool that can add interactivity and responsiveness to your games. They add to the fine-tuning and detail of a game's look and animation

- We've learned how to use Unity's particle system in the previous lab. This time, we'll use animation – we'll use a sprite sheet to swipe one sprite with another to create the illusion of animating a particle effect

Exercise #17 – Particle Effect (Animated) (Cont.)

- Add a Brick game object to the scene and place it on a suitable Sorting Layer, then add a BoxCollider 2D, and a script. Name it *BrickBreak*





Exercise #17 – Particle Effect (Animated) (Cont.)

- Create an instance of the SpriteRenderer component, and a variable of type Sprite. Call it explodedBlock

```
// Start is called before the first frame update
// Used to change the sprite
private SpriteRenderer sr;

// The sprite to change into
public Sprite explodedBlock;

// Use this for initialization
void Start()
{
    sr = GetComponent<SpriteRenderer>();
}
```

Exercise #17 – Particle Effect (Animated) (Cont.)

- We'll use an *OnCollisionEnter2D* function for this one, and the argument is of type collision
- In the If-condition, we check to see if the game object has the correct tag, and make sure he only needs to hit that block **once**, hence the index number zero

```
// Called when something hits the BrickBlock
void OnCollisionEnter2D(Collision2D other)
{
    // Check if the collision hit the bottom of the block
    if(other.gameObject.tag == "Player" && other.GetContact(0).point.y < transform.position.y)
    {
        // Change the Block sprite
        sr.sprite = explodedBlock;

        // Wait a fraction of a second and then destroy the BrickBlock
        Object.Destroy(gameObject, .2f);
    }
}
```

Exercise #17 – Particle Effect (Animated) (Cont.)

- Logically, the player's y value should always be *smaller* than the block's, so we put that in the condition as well

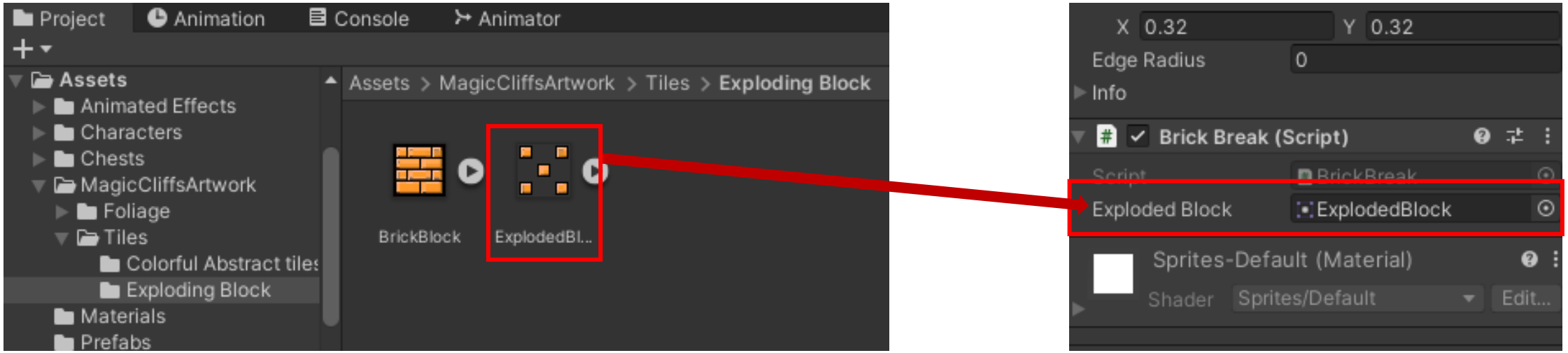
```
// Check if the collision hit the bottom of the block
if(other.gameObject.tag == "Player" && other.GetContact(0).point.y < transform.position.y)
{
```

- If both conditions are true, the Sprite Renderer will replace the BrickBlock sprite with the ExplodedBlock sprite
- Then the block will be destroyed altogether after a fraction of time

```
// Wait a fraction of a second and then destroy the BrickBlock
Object.Destroy(gameObject, .2f);
```

Exercise #17 – Particle Effect (Animated) (Cont.)

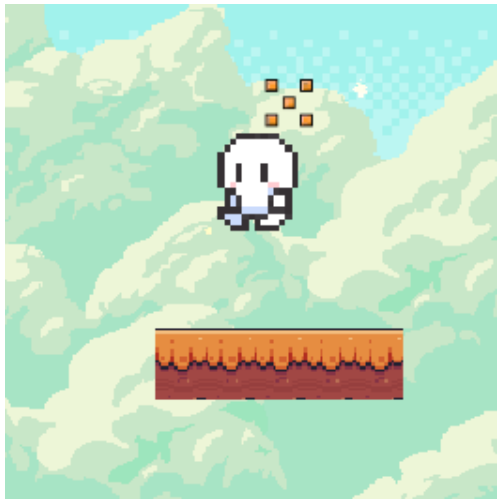
- Go back to Unity and drag and drop your ExplodedBlock sprite to its variable.

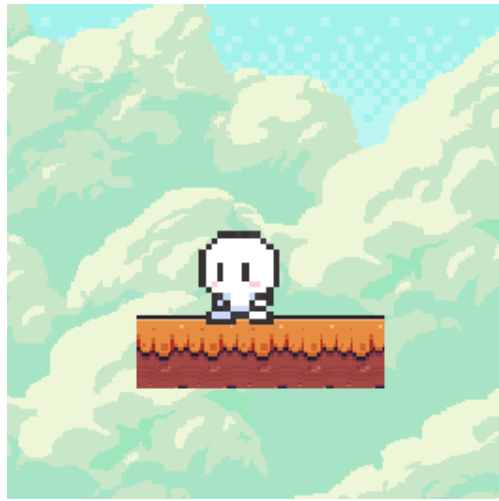


Exercise #17 – Particle Effect (Animated) (Cont.)

- Now when the player game object hits the brick game object from below, it will result in the brick exploding, then disappearing

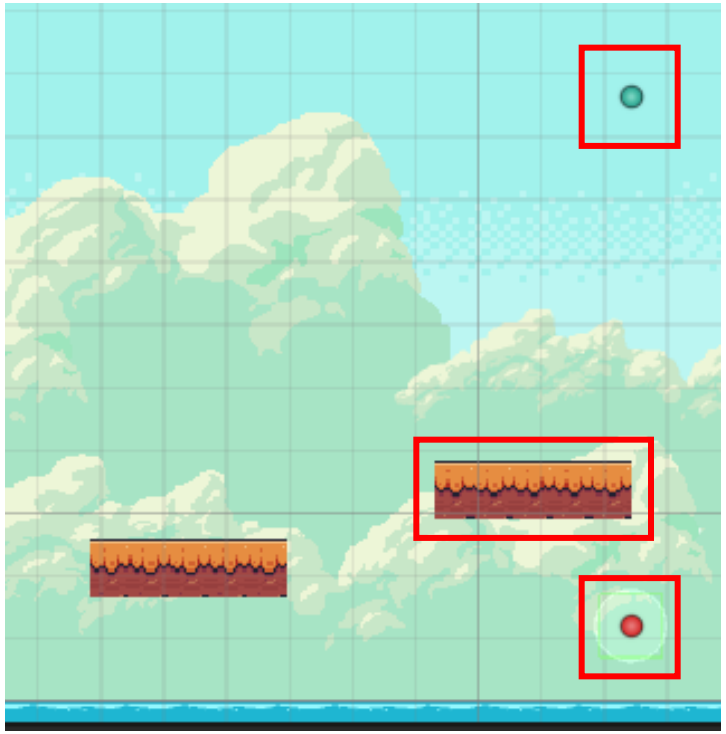






Optional Exercise – Moving Platform

- We want to create a platform that moves back and forth on the y axis between 2 points
- Add a new platform to your scene, and add 2 empty game objects. Name them Point A and Point B



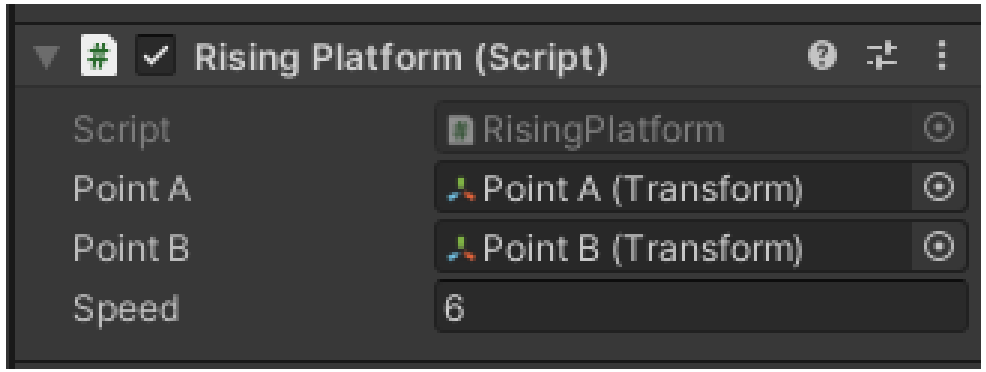
Optional Exercise – Moving Platform (Cont.)

- Add a script to the platform and name it *RisingPlatform*

```
public class RisingPlatform : MonoBehaviour
{
    //platform will move continuously between these 2 points:
    public Transform PointA;
    public Transform PointB;

    public float speed;
    private bool AtoB;

    private Vector3 targetPosition;
```



Optional Exercise – Moving Platform (Cont.)

- The update function will be used to constantly check if the platform is moving in the right direction, and how far is it away from the target position to then change and begin moving in the opposite direction

```
void Update()
{
    //The platform should always start its journey from Point A to Point B.
    //Once AtoB eventually becomes false, the platform will begin moving back towards Point A
    if (AtoB == true)
    {
        targetPosition = PointA.position;
    }
    else
    {
        targetPosition = PointB.position;
    }

    //Move towards target position on Y-axis
    Vector3 newPosition = transform.position;
    newPosition.y = Mathf.MoveTowards(transform.position.y, targetPosition.y, speed * Time.deltaTime);

    transform.position = newPosition;

    //Switch direction when closing in on target
    if (Mathf.Abs(transform.position.y - targetPosition.y) < 0.1f)
    {
        AtoB = !AtoB; //if true, become false. If false, become true
    }
}
```

Optional Exercise – Moving Platform (Cont.)

- But now we have a problem: when the player hops onto the platform, physics elements such as mass and gravity will interrupt the platform's smooth movement! So an easy solution is to make the player a child of the platform as soon as he lands on it. It can be done with an *OnCollisionEnter2D* function:

```
//To make the character sit still on the platform, we will make him a child as soon as he stands
//on it
void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.gameObject.tag=="Player")
    {
        //Make the platform a parent
        collision.transform.SetParent(transform);
    }
}
```

Optional Exercise – Moving Platform (Cont.)

- See how the Player became a child in the Hierarchy as soon as he landed on the platform?



Optional Exercise – Moving Platform (Cont.)

- And when the player hops off, we want everything to return to normal, and the player is no longer a child of the platform. We'll use an *OnCollisionExit2D* function

```
//The moment the character jumps, he's no longer a child of the platform
void OnCollisionExit2D(Collision2D collision)
{
    if (collision.gameObject.tag=="Player")
    {
        //Unparent the player
        collision.transform.SetParent(null);
    }
}
```